JavaScript & HTTP Actions

> **Series:** WFLOW | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Custom Code and API Integration

When built-in actions aren't enough, use JavaScript and HTTP requests for custom integrations. This notebook covers the JavaScript SDK, HTTP request patterns, and common integration scenarios.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write`
Prior Knowledge	**WFLOW-01** through **WFLOW-07**, JavaScript basics

1. JavaScript Action Basics

Action Structure

```
// Every JavaScript action must export a default async function
export default async function(context) {
  // context contains:
  // - event(): trigger event data
  // - result(taskName): previous task results
  // - env: environment secrets

  // Your logic here
  const result = await doSomething();

  // Return value becomes task result
  return {
    output: result,
    success: true
  };
}
```

Accessing Context

```
export default async function({ event, result, env }) {
  // Trigger event data
  const problemTitle = event().title;
  const severity = event().severity;

  // Previous task result
  const previousOutput = result('previous_task').data;

  // Environment secrets
  const apiKey = env.API_KEY;

  return { title: problemTitle };
}
```

Task Configuration

```
name: custom_javascript
type: dynatrace.automations:run-javascript
input:
  script: |
    export default async function({ event }) {
      return { processed: true };
    }
```

2. Dynatrace SDK Usage

Available SDK Clients

Client	Purpose	Import
`queryExecutionClient`	Execute DQL queries	`@dynatrace-sdk/client-query`
`entitiesClient`	Entity CRUD operations	`@dynatrace-sdk/client-classic-environment-v2`
`problemsClient`	Problem management	`@dynatrace-sdk/client-classic-environment-v2`
`eventsClient`	Event ingestion	`@dynatrace-sdk/client-classic-environment-v2`
`settingsClient`	Settings management	`@dynatrace-sdk/client-classic-environment-v2`

Execute DQL Query

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch logs, from: now() - 1h
        | filter loglevel == "ERROR"
        | summarize error_count = count()
      `,
      requestTimeoutMilliseconds: 30000
    }
  });

  const records = result.result.records || [];
  const errorCount = records[0]?.error_count || 0;

  return { error_count: errorCount };
}
```

Get Entity Details

```
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-
v2';

export default async function({ event }) {
  const entityId = event().root_cause_entity_id;

  const entity = await entitiesClient.getEntity({
    entityId: entityId,
    fields: '+properties,+tags'
  });

  return {
    name: entity.displayName,
    type: entity.type,
    tags: entity.tags || []
  };
}
```

Add Problem Comment

```
import { problemsClient } from '@dynatrace-sdk/client-classic-environment-
v2';

export default async function({ event, result }) {
  const problemId = event().display_id;
  const actionTaken = result('remediation').action;

  await problemsClient.createComment({
    problemId: problemId,
    body: {
      message: `[Automated] Remediation executed: ${actionTaken}`,
      context: 'Workflow automation'
    }
  });

  return { comment_added: true };
}
```

3. HTTP Request Patterns

HTTP Request Action

```
name: call_external_api
type: dynatrace.http:request
input:
  url: "https://api.example.com/endpoint"
  method: POST
  headers:
    Authorization: "Bearer {{ env.API_TOKEN }}"
    Content-Type: "application/json"
  body: |
    {
      "problem_id": "{{ event()['display_id'] }}",
      "severity": "{{ event()['severity'] }}",
      "title": "{{ event()['title'] }}"
    }
```

Using fetch() in JavaScript

```
export default async function({ event, env }) {
  const response = await fetch('https://api.example.com/webhook', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${env.API_TOKEN}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      problem_id: event().display_id,
      severity: event().severity,
      title: event().title,
      timestamp: new Date().toISOString()
    })
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${await response.text()}`);
  }

  const data = await response.json();
  return { response: data };
}
```

GET Request with Query Parameters

```
export default async function({ event, env }) {
  const params = new URLSearchParams({
    entity_id: event().root_cause_entity_id,
    from: new Date(Date.now() - 3600000).toISOString(),
    to: new Date().toISOString()
  });

  const response = await fetch(
    `https://api.example.com/metrics?${params}`,
    {
      headers: {
        'Authorization': `Bearer ${env.API_TOKEN}`
      }
    }
  );

  return await response.json();
}
```

4. Error Handling

Try-Catch Pattern

```
export default async function({ event, env }) {
  try {
    const response = await fetch('https://api.example.com/action', {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${env.TOKEN}` },
      body: JSON.stringify({ id: event().display_id })
    });

    if (!response.ok) {
      const errorText = await response.text();
      return {
        success: false,
        error: `HTTP ${response.status}`,
        details: errorText
      };
    }

    return {
      success: true,
      data: await response.json()
    };

  } catch (error) {
    return {
      success: false,
      error: error.message,
      stack: error.stack
    };
  }
}
```

Retry Logic

```
async function fetchWithRetry(url, options, maxRetries = 3) {
  let lastError;

  for (let attempt = 1; attempt &lt;= maxRetries; attempt++) {
    try {
      const response = await fetch(url, options);

      if (response.ok) {
        return response;
      }

      // Retry on 5xx errors
      if (response.status &gt;= 500 &amp;&amp; attempt &lt; maxRetries) {
        await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
        continue;
      }

      throw new Error(`HTTP ${response.status}`);

    } catch (error) {
      lastError = error;
      if (attempt &lt; maxRetries) {
        await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
      }
    }
  }

  throw lastError;
}

export default async function({ event, env }) {
  const response = await fetchWithRetry(
    'https://api.example.com/action',
    {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${env.TOKEN}` }
    }
  );

  return await response.json();
}
```

5. Working with Data

Transform DQL Results

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch logs, from: now() - 1h
        | filter loglevel == "ERROR"
        | fields timestamp, content, dt.entity.service
        | limit 100
      `
    }
  });

  const records = result.result.records || [];

  // Group by service
  const byService = records.reduce((acc, log) =&gt; {
    const service = log['dt.entity.service'] || 'unknown';
    if (!acc[service]) {
      acc[service] = [];
    }
    acc[service].push(log.content);
    return acc;
  }, {});

  // Create summary
  const summary = Object.entries(byService).map(([service, logs]) =&gt; ({
    service,
    error_count: logs.length,
    sample: logs[0]
  }));

  return {
    total_errors: records.length,
    by_service: summary
  };
}
```

Parse Event Tags

```
export default async function({ event }) {
  const tags = event().tags || [];

  // Parse key:value tags
  const tagMap = tags.reduce((acc, tag) =&gt; {
    if (tag.includes(':')) {
      const [key, value] = tag.split(':');
      acc[key] = value;
    } else {
      acc[tag] = true;
    }
    return acc;
  }, {});

  return {
    team: tagMap['team'] || 'unknown',
    env: tagMap['env'] || 'unknown',
    is_critical: tagMap['critical'] || false
  };
}
```

Build Dynamic Queries

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  const entityId = event().root_cause_entity_id;
  const severity = event().severity;

  // Build query based on entity type
  let query;
  if (entityId.startsWith('SERVICE-')) {
    query = `
      fetch spans, from: now() - 1h
      | filter dt.entity.service == "${entityId}"
      | filter span.status_code &gt;= 500
      | summarize errors = count()
    `;
  } else if (entityId.startsWith('HOST-')) {
    query = `
      fetch logs, from: now() - 1h
      | filter dt.entity.host == "${entityId}"
      | filter loglevel == "ERROR"
      | summarize errors = count()
    `;
  }

  const result = await queryExecutionClient.queryExecute({
    body: { query }
  });

  return {
    entity_type: entityId.split('-')[0],
    errors: result.result.records[0]?.errors || 0
  };
}
```

6. Integration Examples

GitHub Issue Creation

```
export default async function({ event, env }) {
  const response = await fetch(
    `https://api.github.com/repos/${env.GITHUB_REPO}/issues`,
    {
      method: 'POST',
      headers: {
        'Authorization': `token ${env.GITHUB_TOKEN}`,
        'Accept': 'application/vnd.github.v3+json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title: `[Dynatrace] ${event().title}`,
        body: `
&lt;a id="problem-details"&gt;&lt;/a&gt;
## Problem Details
- **Problem ID:** ${event().display_id}
- **Severity:** ${event().severity}
- **Started:** ${event().start_time}
- **Link:** [View in Dynatrace](${event().problem_url})

&lt;a id="affected-entities"&gt;&lt;/a&gt;
## Affected Entities
${event().affected_entity_ids.map(e =&gt; `- ${e}`).join('\n')}
        `,
        labels: ['dynatrace', event().severity.toLowerCase()]
      })
    }
  );

  const issue = await response.json();
  return { issue_number: issue.number, url: issue.html_url };
}
```

Datadog Event

```
export default async function({ event, env }) {
  const response = await fetch('https://api.datadoghq.com/api/v1/events', {
    method: 'POST',
    headers: {
      'DD-API-KEY': env.DATADOG_API_KEY,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      title: event().title,
      text: `Problem ${event().display_id} detected by Dynatrace`,
      alert_type: event().severity === 'CRITICAL' ? 'error' : 'warning',
      source_type_name: 'dynatrace',
      tags: ['source:dynatrace', `severity:${event().severity}`]
    })
  });

  return await response.json();
}
```

Splunk HEC Event

```
export default async function({ event, env }) {
  const response = await fetch(
    `${env.SPLUNK_HEC_URL}/services/collector/event`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Splunk ${env.SPLUNK_HEC_TOKEN}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        event: {
          problem_id: event().display_id,
          title: event().title,
          severity: event().severity,
          affected_entities: event().affected_entity_ids
        },
        sourcetype: 'dynatrace:problem',
        index: 'observability'
      })
    }
  );

  return { sent: response.ok };
}
```

7. Performance Tips

Parallel Execution

```
export default async function({ event }) {
  const entityIds = event().affected_entity_ids;

  // Execute all lookups in parallel
  const results = await Promise.all(
    entityIds.map(id =&gt; getEntityDetails(id))
  );

  return { entities: results };
}
```

Limit Query Scope

```
// Bad: Fetching too much data
const result = await queryExecutionClient.queryExecute({
  body: {
    query: `fetch logs, from: now() - 7d | filter loglevel == "ERROR"`
  }
});

// Good: Limited time range and fields
const result = await queryExecutionClient.queryExecute({
  body: {
    query: `
      fetch logs, from: now() - 1h
      | filter loglevel == "ERROR"
      | fields timestamp, content
      | limit 100
    `,
    requestTimeoutMilliseconds: 30000
  }
});
```

Use Timeouts

```
// Add timeout to external calls
const controller = new AbortController();
const timeout = setTimeout(() =&gt; controller.abort(), 10000);

try {
  const response = await fetch(url, {
    signal: controller.signal,
    // ... other options
  });
  return await response.json();
} finally {
  clearTimeout(timeout);
}
```

Monitor JavaScript Task Performance

```
// JavaScript task execution times
fetch events, from: now() - 7d
| filter event.type == "automation.task.execution"
| filter contains(task.type, "javascript")
| summarize
    avg_duration = avg(task.duration),
    max_duration = max(task.duration),
    p95_duration = percentile(task.duration, 95),
    total = count(),
    by:{workflow.name, task.name}
| sort avg_duration desc
| limit 20
```

```
// HTTP request task errors
fetch events, from: now() - 24h
| filter event.type == "automation.task.execution"
| filter contains(task.type, "http") or contains(task.type, "javascript")
| filter task.status == "FAILED"
| fields timestamp, workflow.name, task.name, task.error
| sort timestamp desc
| limit 20
```

Next Steps

With custom integrations ready, learn governance:

Recommended Path

1. **WFLOW-09: Security, Governance & Monitoring** - Production best practices

Key Takeaways

- **JavaScript actions** enable custom logic
- **Dynatrace SDK** provides typed API access
- **HTTP requests** integrate any REST API
- **Error handling** is critical for reliability
- **Performance** matters - limit query scope, use parallelism

Summary

In this notebook, you learned:

- JavaScript action structure and context
- Dynatrace SDK clients and methods
- HTTP request patterns (GET, POST, auth)
- Error handling and retry logic
- Data transformation patterns
- Integration examples (GitHub, Datadog, Splunk)
- Performance optimization tips

References

- JavaScript Action
- HTTP Request Action
- Dynatrace SDK
- DQL Reference
